Cyberscope

A TAC Security Company

Audit Report

Phoenix Token

September 2026

Repository	https://github.com/phoenixtoken-pxt/phoenix-protocol/
Commit	a8adc8f96a12a0ffae247a09cb0ae2e9eb435b73
Audited by	© cyberscope

Table of Contents

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation**: This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation**: This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical**: Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium**: Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor**: Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative**: Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
Critical	Highly Likely / High Impact
Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
Minor / Informative	Unlikely / Low to no Impact

Review

Audit Updates

Initial Audit	30 Jul 2026 https://github.com/cyberscope-io/audits/blob/main/2-pxt/v1/audit.pdf
Corrected Phase 2	28 Aug 2026 https://github.com/cyberscope-io/audits/blob/main/2-pxt/v2/audit.pdf
Corrected Phase 3	03 Sep 2026

Source Files

Filename	SHA256
src/uniswap/v4/V4Addresses.sol	140f35c3f855b9cc91e7982d32e06a1d4b4c6ce84103a033ef42a11a34c4521d
src/uniswap/v4/HookMiner.sol	45821cd13e0fc55bf1902cdb2d6b8b2f47fe5e17529fb008f15229042004e22b
src/return-delta/PhoenixV4ReturnDeltaHook.sol	62af56229e2b8d1f6bd07334fdf16c688620ae4feca2067ce7635aa0ae8f7071
src/return-delta/ISellAttributor.sol	28eb91001fafc7d2e403e09d95848361c75569441a23b76bb67f1b8cf1fe8fe4
src/fee/PhoenixFeeCollector.sol	c56c34049a75c282f551d5d75f9e5a87c7c29747fdd988181f0b9bff152b1ea4
src/fee/PhoenixBuybackMath.sol	68ce58e39f54f97f1cfd8944ab4146979d28ea0fa5c9182ebde6c243a9d24473

src/fee/PhoenixBuyback.sol	0106879b58d9a7da789bb018bfe806c5125dfc85a44f0ed80bb8efe122437efb
src/core/PxtSellAccess.sol	ccad99715e3c49fd57fdbe201275035c2fd98559714c91f70b988b9a9f914da8
src/core/PxtFeeModel.sol	19044ddbe9a049241851a2b1e83c4218150d31030c36beb054666c2c922810c1
src/core/Pxt.sol	2e8d87eda4c4fb5c12051d927c036c6fbba39254530f164dd06ed5df596d671b

Overview

Phoenix Token is a Base-focused token ecosystem built around PXT, a fixed-supply, six-decimal ERC-20 integrated with a Uniswap v4 ReturnDelta hook. The protocol applies transfer, buy, sell, and dump-penalty fees; enforces a timed sell lock and anti-bot opening ceremony; restricts transfers to unapproved contracts; collects fees in USDC; burns part of sold PXT; and uses accumulated buyback fees to purchase PXT and recycle it into single-sided liquidity positions.

The intended production PXT/USDC PoolKey is configured with a 0% native Uniswap LP fee. Protocol fees are instead imposed through the token and hook's ReturnDelta accounting. The zero LP fee is established by the deployment configuration and is not independently enforced by the contracts' pool-configuration functions.

Pxt.sol

`Pxt` implements the fixed-supply 40.021-trillion Phoenix Token with ERC-20 Permit support. It charges a 2.7% wallet-transfer fee, enforces the sell lock and anti-bot controls, manages special wallet statuses and approved contract recipients, supports token burning, and reports authenticated DEX settlements to the configured sell-attribution hook.

PxtFeeModel.sol

`PxtFeeModel` defines the protocol's fee types, constants, errors, and pure calculation functions. It divides buy and transfer fees between donation and marketing wallets, calculates the fixed PXT burn on sells, and splits normal or penalty USDC sell fees among donation, marketing, and buyback obligations.

PxtSellAccess.sol

`PxtSellAccess` provides shared checks for the immutable sell-unlock timestamp and anti-bot state. It distinguishes between seller-specific ERC-20 settlement checks and the general "trading open" condition used by the hook and fee collector.

PhoenixFeeCollector.sol

`PhoenixFeeCollector` is the protocol treasury that receives and accounts for fees skimmed by the hook. Its permissionless `collect()` operation pays pending donation and marketing obligations, burns available PXT, reconciles underfunded balances, and preserves the USDC buyback allocation for authorized buyback execution.

The collector does not pull native Uniswap LP fees from protocol liquidity positions. The intended production pool uses a 0% native Uniswap LP fee, so no such LP fee growth is expected.

PhoenixBuyback.sol

`PhoenixBuyback` supplies the fee collector's seed-liquidity and buyback machinery. It owns the protocol's seed position, allows authorized executors to swap accrued USDC for PXT subject to deadline and slippage controls, and deposits purchased PXT into single-sided recycle positions constructed from the current pool tick.

Buyback pricing and execution limits use the last promoted official-pool spot observation from a prior block. The production pool's native Uniswap LP fee is configured as zero, and collector-originated buybacks are exempted from the hook's buy fee.

PhoenixBuybackMath.sol

`PhoenixBuybackMath` contains the pure arithmetic used by the buyback system. It converts quote-token value into expected PXT output, derives price limits, enforces minimum output, aligns ticks to pool spacing, and constructs and validates PXT-only recycle ranges.

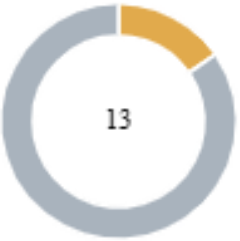
PhoenixV4ReturnDeltaHook.sol

`PhoenixV4ReturnDeltaHook` is the Uniswap v4 hook for the official PXT/USDC pool. The intended production PoolKey uses a 0% native Uniswap LP fee; protocol buy, sell, burn, and dump-penalty fees are imposed separately through ReturnDelta accounting.

The hook skims USDC on buys and sells, coordinates PXT burns, applies the fixed 24-hour per-address dump-penalty window, rebates eligible sellers after token-authenticated attribution, handles orphaned ERC-6909 settlements, gates liquidity during the sell lock,

records official-pool spot observations for buyback execution, and forwards finalized fees to the collector.

Findings Breakdown



Critical	0
Medium	2
Minor / Informative	11

Severity	Unresolved	Acknowledged	Resolved	Other
Critical	0	0	0	0
Medium	0	2	0	0
Minor / Informative	0	11	0	0

Diagnostics

Critical Medium Minor / Informative

Severity	Code	Description	Status
Medium	PTTB	PoolManager Transfer Tax Bypass	Acknowledged
Medium	VEEO	Value Extraction Enabling Operations	Acknowledged
Minor / Informative	APTE	Alternate Pool Tax Evasion	Acknowledged
Minor / Informative	BIBP	Balance Inflation Bypasses Penalty	Acknowledged
Minor / Informative	CCIB	Code-Length Check Is Bypassable	Acknowledged
Minor / Informative	CCR	Contract Centralization Risk	Acknowledged
Minor / Informative	FWEPE	Fixed Windows Enable Penalty Evasion	Acknowledged
Minor / Informative	FAPR	Fresh Address Penalty Reset	Acknowledged
Minor / Informative	MC	Missing Check	Acknowledged
Minor / Informative	OPIIU	Official Pool Initialization Is Unrestricted	Acknowledged
Minor / Informative	OSLP	Orphaned Sell Locks PXT	Acknowledged
Minor / Informative	STCBT	Seed Transfers Can Be Taxed	Acknowledged
Minor / Informative	VPBB	Valid Price Breaks Buybacks	Acknowledged

PTTB - PoolManager Transfer Tax Bypass

Criticality	Medium
Location	Pxt.sol#L223,325
Status	Acknowledged

Description

PXT restricts transfers to contracts through a recipient allowlist, but treats every outbound transfer from the singleton PoolManager as trusted settlement: `_enforceContractRecipient` short-circuits when `from == poolManager`, skipping the allowlist check entirely, and `_quoteTransfer` returns a zero fee for the same condition. Because Uniswap v4 lets a holder mint ERC-6909 claims against PXT and redeem them to an arbitrary address — and delivers swap/withdraw output the same way — any account can cause the PoolManager to push PXT to a contract that was never approved, and to do so fee-free on the outbound leg. The inbound side is now authenticated (unattested deposits pay the 2.7% transfer fee), so the earlier fee-avoidance round trip is closed; however, the contract-recipient restriction itself can still be sidestepped through PoolManager redemption.

The exemption cannot be removed outright, since the pool must be able to settle legitimate swap and liquidity proceeds; the issue is that settlement authorization is inferred solely from the `poolManager` address rather than from an authenticated operation, so it also covers claim redemptions the protocol did not intend to whitelist.

```
// _enforceContractRecipient
if (poolManager != address(0) && from == poolManager) return; //
allowlist skipped

// _quoteTransfer
if (poolManager != address(0) && from == poolManager) {
    return PxtFeeModel.noFee(amount); // no fee on any PoolManager
outbound
}
```

Recommendation

Distinguish genuine swap and liquidity settlement from arbitrary claim redemptions rather than trusting every transfer originating from the PoolManager. Where the allowlist is intended to constrain which contracts may hold PXT, that constraint should be evaluated on the ultimate recipient of PoolManager-mediated outflows as well, so that redemption cannot be used as a channel to deliver tokens to unapproved contracts. Alternatively, the recipient restriction's security assumptions should be documented to explicitly acknowledge the settlement path as an accepted exception.

Team Update

The team has acknowledged that this is not a security issue and states:

Inbound tax bypass already fixed. Outbound: `from == poolManager` *is fee-free and skips the contract-recipient allowlist (swaps, LP exits, ERC-6909 redeem). v4 settlement cannot be authenticated at the token. The allowlist is a P2P gate, not a hard custody rule.*

VEEO - Value Extraction Enabling Operations

Criticality	Medium
Location	src/return-delt/PhoenixV4ReturnDeltaHook.sol#L244 src/fee/PhoenixBuyback.sol#L193,202,209,309
Status	Acknowledged

Description

Protocol operations that execute predictably against public market state may allow external actors to position themselves before, during, or after the operation and extract value from the protocol.

Value extraction can take multiple forms, including manipulating a price reference, sandwiching a protocol trade, supplying just-in-time liquidity, capturing protocol order flow, or otherwise influencing the market state used to calculate or execute the operation. Restricting execution to an authorized caller reduces who can initiate the operation, but it does not necessarily prevent other market participants from anticipating or influencing it.

For example, in the current implementation, the buyback uses the last official-pool spot price observed in a previous block as both its expected-output reference and the basis of its execution limit. This reference is derived from an individual pool observation rather than a manipulation-resistant average or independent price source.

An attacker may manipulate the final observed pool price in one block and cause that price to become the reference for a buyback in a later block. If the attacker can anticipate the keeper’s execution or maintain the manipulated market state, the protocol may buy PXT at an unfavorable price. The attacker can sell into the protocol’s demand or unwind a position after the buyback, extracting part of the protocol’s USDC budget.

The authorized-caller restriction and previous-block delay make the original permissionless same-transaction attack more difficult, but they do not remove the general value-extraction risk.

```
function _noteOfficialSpot() internal {
    if (address(feeCollector) == address(0)) return;
    (uint160 sqrtPriceX96,,,) = poolManager.getSlot0(officialPoolId);
    if (sqrtPriceX96 != 0) {
        feeCollector.noteOfficialSpot(sqrtPriceX96);
    }
}
...
function _promoteBuybackSpot() internal {
    if ( pendingSpotBlock != 0 && block.number > pendingSpotBlock ) {
        frozenSqrtPriceX96 = pendingSqrtPriceX96;
    }
}
...
uint160 sqrtPriceX96 = _buybackRefSqrtPriceX96();
...
uint160 sqrtLimit =
PhoenixBuybackMath.sqrtPriceLimitAfterSlippage(sqrtPriceX96, zeroForOne,
maxBuybackSlippageBps);
...
uint256 expectedPxt = PhoenixBuybackMath.pxtForQuote(usdcSpent,
sqrtPriceX96, pxtIsToken0);
```

Recommendation

Consider designing protocol-controlled market operations so that external actors cannot profitably influence their pricing inputs, execution conditions, timing, or surrounding liquidity.

The mitigation should address the general value-extraction and MEV surface Relevant mechanisms may include manipulation-resistant price references, sufficiently long or robust averaging periods, independent price validation, bounded trade sizes, execution-price deviation checks, private or protected execution, randomized or less predictable execution, auctions, batching, or other mechanisms appropriate to the protocol's liquidity and operational model.

Team Update

The team has acknowledged that this is not a security issue and states:

Same residual as JLVE. Buyback is keeper-only and limited vs previous-block official spot (2%). Remaining MEV is public-mempool sandwich inside that band. Ops: private relay, quote-based `minPxtBought`, clip vs depth.

APTE - Alternate Pool Tax Evasion

Criticality	Minor / Informative
Location	Pxt.sol#L249 PhoenixV4ReturnDeltaHook.sol#L260
Status	Acknowledged

Description

The 5.4% base sell fee and the up-to-37.8% dump penalty (including the PXT burn and USDC skim) are enforced only for swaps routed through the official Phoenix hook, which sets the transient `pendingDexSellAmount()` marker consumed during settlement. `Pxt._update` still recognizes any transfer whose destination is the singleton PoolManager as a candidate DEX interaction, and authenticates a taxed sell solely by that marker. An attacker can initialize a separate PXT/quote pool on the same PoolManager with no hook attached and sell into it: the official hook never runs, so `pendingDexSellAmount()` returns zero, `consumeLpInbound` finds no reserved LP inbound, and the settlement falls through to the ordinary wallet-transfer branch.

Consequently the seller moves PXT into the unofficial pool paying only the 2.7% transfer fee, while `_onDexSell` / `attributeSell` never execute — no sell fee, no dump penalty, and no burn are applied. Any holder can thereby liquidate an arbitrarily large position at the reduced transfer rate and avoid the sell/penalty tiers the protocol relies on, draining fee revenue and defeating dump protection.

```
if (address(sellAttributor) != address(0)) {
    uint256 pendingSell = sellAttributor.pendingDexSellAmount();
    if (pendingSell != 0) {
        // Real DEX sell, or a mismatched settle that must revert in
attributeSell.
        _enforceSellAccess(from);
        super._update(from, to, amount);
        _onDexSell(from, amount);
        return;
    }
    if (sellAttributor.consumeLpInbound(amount)) {
        super._update(from, to, amount);
        return;
    }
    // Fall through: 2.7% wallet tax.
}
```

Recommendation

Bind privileged token settlement to authenticated activity in the official pool.

Team Update

The team has acknowledged that this is not a security issue and states:

Hook fees apply only to the official pool. A no-hook v4 pool on the same PoolManager settles as a 2.7% transfer. Permissionless v4 cannot bind every pool at the token. Mitigation is liquidity + routing on the official pool.

BIBP - Balance Inflation Bypasses Penalty

Criticality	Minor / Informative
Location	src/core/Pxt.sol#L240 src/return-delta/PhoenixV4ReturnDeltaHook.sol#L251,560,581,597
Status	Acknowledged

Description

The dump-penalty threshold depends on the seller's balance immediately before the sale. A balance observed at a single point in time does not necessarily represent tokens economically owned by the seller because tokens can be transferred, borrowed, or otherwise controlled temporarily.

The implementation excludes PXT transferred directly from PoolManager to the seller during the same transaction. It also reduces an existing window's balance snapshot when a later sale observes that the seller's holdings have decreased.

These controls address some direct PoolManager-based inflation scenarios, but they do not establish that the remaining balance is permanently or economically owned by the seller. PXT received temporarily from another account, contract, lender, or intermediary is not identified by _effectiveSellBalance().

A user can therefore temporarily increase the selling address's balance, execute a sale while that inflated balance is used as the penalty denominator, and return the temporary tokens afterward. The downward ratchet does not prevent the initial avoidance because it only observes the lower balance during a subsequent sale.

```
function creditFromPoolManager( address account, uint256 amount )
external {
    if (msg.sender != address(pxt)) revert OnlyPxt();
    if (account == address(0) || amount == 0) return;
    bytes32 slot = _pmInSlot(account);
    _tstore(slot, _tload(slot) + amount);
}
...
function _effectiveSellBalance( address seller, uint256 pxtAmount )
internal view returns (uint256) {
    uint256 raw = pxt.balanceOf(seller) + pxtAmount;
    uint256 fromPm = _tload(_pmInSlot(seller));
    if (raw > fromPm) {
        return raw - fromPm;
    }
    return pxtAmount;
}
...
uint256 soldInWindow = window.soldInWindow;
...
uint256 balanceAtStart = window.balanceAtWindowStart;
if ( window.windowStart == 0 || block.timestamp >= window.windowStart +
PxtFeeModel.PENALTY_WINDOW ) {
    soldInWindow = 0;
    balanceAtStart = balanceBefore;
}
...
if (balanceAtStart > balanceBefore) {
    balanceAtStart = balanceBefore;
}
...
uint256 newSold = soldInWindow + amount;
...
bool penalized = balanceAtStart > 0 && newSold * PxtFeeModel.BPS >
balanceAtStart * PxtFeeModel.PENALTY_THRESHOLD_BPS;
```

Recommendation

Consider evaluating whether a balance-based dump penalty can reliably represent economic ownership in the protocol's intended environment.

Any mitigation should address the general ability to obtain or distribute temporary balances rather than only transfers originating directly from a known contract. Possible designs may use historical or time-weighted balances, transfer-aware holding periods, delayed eligibility,

conservative balance checkpoints, cooldowns, account-level quotas, or other mechanisms that make temporary ownership less useful.

Team Update

The team has acknowledged that this is not a security issue and states:

Residual of FBBP. Same-tx PoolManager padding is excluded. Other inbound (EOA / lender) still counts; that path pays 2.7% both ways. Dump quota is pre-sell balance per address, not economic ownership.

CCIB - Code-Length Check Is Bypassable

Criticality	Minor / Informative
Location	Pxt.sol#L198,206
Status	Acknowledged

Description

`_isWallet()` classifies any address with no deployed bytecode as a wallet, allowing `_enforceContractRecipient()` to bypass the contract-recipient allowlist. However, zero code does not prove that an address is permanently an EOA: an attacker can precompute a future CREATE2 address, transfer PXT to it while `code.length` is zero, and deploy a contract at that address afterward.

The newly deployed, unapproved contract retains full control over the prefunded PXT and can transfer or otherwise use it, defeating the intended restriction on sending tokens to contracts. The same underlying limitation can affect addresses during contract construction.

```
function _isWallet(address account) internal view returns (bool) {
    uint256 size = account.code.length;
    if (size == 0) return true;
    if (size != 23) return false;
    bytes memory code = account.code;
    return code[0] == 0xef && code[1] == 0x01 && code[2] == 0x00;
}

if (_isWallet(to)) return;
if (isApprovedContractRecipient[to]) return;
revert ContractRecipientNotApproved();
```

Recommendation

The inherent inability to distinguish EOAs from undeployed contract addresses should be accounted for in the restriction’s security assumptions.

Team Update

The team has acknowledged that this is not a security issue and states:

23-byte path already closed (`_isWallet` is `code.length == 0` only). CREATE2 / constructor residual stands: zero code cannot prove a permanent EOA. Impact is routing tokens to a self-chosen contract, not a protocol drain.

CCR - Contract Centralization Risk

Criticality	Minor / Informative
Location	Pxt.sol#L111,123,143,151,161,167 PhoenixV4ReturnDeltaHook.sol#L112,120,126 PhoenixFeeCollector.sol#L27 PhoenixBuyback.sol#L167,180
Status	Acknowledged

Description

The contract's functionality and behavior are heavily dependent on external parameters or configurations. While external configuration can offer flexibility, it also poses several centralization risks that warrant attention. Centralization risks arising from the dependence on external configuration include Single Point of Control, Vulnerability to Attacks, Operational Delays, Trust Dependencies, and Decentralization Erosion.

```
function setPoolManager(address poolManager_) external onlyOwner;
function setAntiBotSeller(address seller) external onlyOwner;
function setFeeCollector(address collector_) external onlyOwner;
function setSellAttributor(ISellAttributor attributor_) external
onlyOwner;
function setWalletStatus(address wallet, WalletStatus status) external
onlyOwner;

function setApprovedContractRecipient(address recipient, bool approved)
    external
    onlyRole(RECIPIENT_APPROVER_ROLE);

function setFeeCollector(PhoenixFeeCollector collector_) external
onlyOwner;
function setLiquidityProvider(address account, bool allowed) external
onlyOwner;
function setOfficialPool(PoolKey calldata key) external onlyOwner;

function setHook(address hook_) external onlyOwner;
function configurePool(
    PoolKey calldata key,
    int24 tickLower_,
    int24 tickUpper_,
    bytes32 salt_
) external onlyOwner;

function setBuybackParams(
    uint24 recycleWidthSpacings_,
    uint16 maxBuybackSlippageBps_
) external onlyOwner;
```

Recommendation

To address this finding and mitigate centralization risks, it is recommended to evaluate the feasibility of migrating critical configurations and functionality into the contract's codebase itself. This approach would reduce external dependencies and enhance the contract's self-sufficiency. It is essential to carefully weigh the trade-offs between external configuration flexibility and the risks associated with centralization.

Team Update

The team has acknowledged that this is not a security issue and states:

Owner setters are bootstrap. `LockProtocolReturnDelta` *renounces Ownable on FeeCollector → Hook → Pxt before go-live. Post-lock: Safe may only manage contract recipients and buyback callers.* `setLiquidityProvider` *is gone.*

FWEPE - Fixed Windows Enable Penalty Evasion

Criticality	Minor / Informative
Location	PhoenixV4ReturnDeltaHook.sol#L466,486
Status	Acknowledged

Description

The dump protection uses a fixed 24-hour bucket beginning with the seller's first sale rather than a rolling 24-hour lookback. A seller can use a dust sale to choose the window boundary, sell the remainder of their approximately 10% allowance immediately before expiry, and sell another 10% of their new starting balance immediately after the reset. This allows roughly 19% of the original position to be sold seconds apart without triggering the dump penalty.

This attack is independent of the balance-snapshot inflation variant: it requires no temporary liquidity or additional wallets. The hard reset discards all prior sales at the boundary, even when they occurred only seconds earlier, while the new sale establishes a fresh balance denominator.

```
if (
    window.windowStart == 0
        || block.timestamp >= window.windowStart +
PxtFeeModel.PENALTY_WINDOW
) {
    soldInWindow = 0;
    balanceAtStart = balanceBefore;
}

uint256 newSold = soldInWindow + amount;
bool penalized =
    balanceAtStart > 0
        && newSold * PxtFeeModel.BPS
            > balanceAtStart * PxtFeeModel.PENALTY_THRESHOLD_BPS;
```

Recommendation

Consider using a rolling 24-hour accounting mechanism that retains sales occurring within the actual lookback period instead of resetting all history at a fixed boundary. The balance

denominator should also be based on a manipulation-resistant measure that remains consistent as holdings change.

Team Update

The team has acknowledged that this is not a security issue and states:

Dump window is a fixed 24h bucket from the first sell, not a rolling lookback. Around expiry a seller can clip ~19% of the then-current bag at base fee. Intended UX.

FAPR - Fresh Address Penalty Reset

Criticality	Minor / Informative
Location	PhoenixV4ReturnDeltaHook.sol#L217,466
Status	Acknowledged

Description

Sell-window state is keyed only by the current seller. A holder can sell exactly 10% of a balance, transfer the remainder to a fresh address, and repeat with a new baseline. This allows most of a position to be liquidated at the base tier rather than the intended dump-penalty tier.

```
SellWindow storage window = sellWindows[seller];

uint256 newSold = soldInWindow + amount;
bool penalized =
    balanceAtStart > 0
        && newSold * PxtFeeModel.BPS
            > balanceAtStart * PxtFeeModel.PENALTY_THRESHOLD_BPS;
```

Recommendation

Design the sell quota so recently transferred balances do not obtain a fresh allowance.

Team Update

The team has acknowledged that this is not a security issue and states:

Quota is per seller address. Sell 10% → transfer rest → repeat can leak more than 10% over time; each hop pays 2.7%, and selling 100% of a fresh wallet is still penalty. No transfer-window inheritance.

MC - Missing Check

Criticality	Minor / Informative
Location	PhoenixV4ReturnDeltaHook.sol#L126 PhoenixBuyback.sol#L167,180,449 PhoenixBuybackMath.sol#L71
Status	Acknowledged

Description

The protocol's privileged configuration functions do not fully validate their inputs before persisting them. `setOfficialPool()` only verifies that the pool contains PXT and does not ensure that `key.hooks` references the current hook. Similarly, the one-shot `configurePool()` does not verify that the key matches the official pool and configured hook, or that its currencies, tick spacing, and position ticks are valid. An incorrect value permanently binds these components to incompatible pools and can disable hook callbacks, liquidity management, fee collection, or buybacks.

Additionally, `setBuybackParams()` only rejects a zero recycle width. Values at or above `2^23` become negative when cast to `int24`, while smaller values can still overflow when multiplied by `tickSpacing`. Consequently, the assumption in `recycleTicks()` is not enforced, and buybacks may consistently revert after ownership has been renounced.

```
function setOfficialPool(PoolKey calldata key) external onlyOwner {
    if (officialPoolSet) revert OfficialPoolAlreadySet();
    if (!_poolContainsPxt(key)) revert InvalidPool();
    officialPoolId = key.toId();
    officialPoolSet = true;
}

function configurePool(PoolKey calldata key, int24 tickLower_, int24
tickUpper_, bytes32 salt_)
    external
    onlyOwner
{
    if (poolConfigured) revert PoolAlreadyConfigured();
    poolKey = key;
    tickLower = tickLower_;
    tickUpper = tickUpper_;
    positionSalt = salt_;
    poolConfigured = true;
}

if (recycleWidthSpacings_ == 0) revert InvalidRecycleWidth();

int24 width = int24(uint24(recycleWidthSpacings)) * spacing;
```

Recommendation

Consider validating that all configured pool keys consistently reference the expected currencies, hook, PoolManager pool, and usable tick parameters before committing one-shot state. The recycle-width limit should account for both the `int24` conversion and multiplication by the configured tick spacing; limiting it only to `type(int24).max` would not prevent multiplication overflow.

Team Update

The team has acknowledged that this is not a security issue and states:

One-shot owner config does not fully cross-check pool keys or cap recycle-width overflow. Wrong values brick that deploy. Bootstrap + lock are the check; no owner after renounce to fix or exploit.

OPIIU - Official Pool Initialization Is Unrestricted

Criticality	Minor / Informative
Location	PhoenixV4ReturnDeltaHook.sol#L142
Status	Acknowledged

Description

The hook does not enable the `beforeInitialize` permission and therefore cannot authorize the initializer or validate the pool's starting price. Once the deterministic hook address and `PoolKey` are known, any account can call the singleton PoolManager and initialize the official pool using an arbitrary valid `sqrtPriceX96`.

Uniswap v4 pool initialization is one-time, so the protocol's subsequent initialization transaction will revert. Recovery may require deploying a different hook or adopting the attacker-selected starting price, which could disrupt deployment and expose initial liquidity if seeding proceeds without additional validation.

```
function getHookPermissions() public pure override returns
(Hooks.Permissions memory) {
    return Hooks.Permissions({
        beforeInitialize: false,
        afterInitialize: false,
        beforeAddLiquidity: true,
        afterAddLiquidity: false,
        beforeRemoveLiquidity: false,
        afterRemoveLiquidity: false,
        beforeSwap: true,
        afterSwap: true,
        beforeDonate: false,
        afterDonate: false,
        beforeSwapReturnDelta: true,
        afterSwapReturnDelta: true,
        afterAddLiquidityReturnDelta: false,
        afterRemoveLiquidityReturnDelta: false
    });
}
```

Recommendation

Consider restricting initialization through an appropriate hook callback that validates the pool key, initializer, and intended starting price. Alternatively, deployment and initialization could be performed atomically so that no external account can initialize the pool first.

Team Update

The team has acknowledged that this is not a security issue and states:

No `beforeInitialize`. Anyone can init a known PoolKey first (grief / wrong spot). Production: initialize in the same tx/bundle as hook reveal; check spot before seed. After init, N/A.

OSLP - Orphaned Sell Locks PXT

Criticality	Minor / Informative
Location	PhoenixV4ReturnDeltaHook.sol#L92,342,647,664
Status	Acknowledged

Description

For an exact-input sell, the hook provisionally takes the burn amount calculated from the full requested PXT input. After execution, it burns only the portion corresponding to the actual fill and stores the remainder in transient storage for refund during `attributeSell()`.

When the sell is settled using ERC-6909 claims, `attributeSell()` is never invoked. The persistent `OrphanSkim` structure does not record either the refundable PXT amount or its beneficiary, and orphan finalization clears the transient value while processing only the USDC skim. The unburned PXT consequently remains held by the hook without an accounting or recovery path.

```
uint256 burnAmount =
    PxtFeeModel.splitSellBurn(PxtFeeModel.SELL_FEE_BPS, pxtAmount);

poolManager.take(pxtCurrency, address(this), burnAmount);
_tstore(_T_SPECIFIED_TAKE, burnAmount);

// After the partial fill:
uint256 taken = _tload(_T_SPECIFIED_TAKE);
uint256 actualBurn =
    PxtFeeModel.grossUp(poolPxt, PxtFeeModel.SELL_BURN_BPS);

_tstore(_T_SPECIFIED_TAKE, taken - actualBurn);

// Orphan finalization clears the only refund record:
_tstore(_T_SPECIFIED_TAKE, 0);
```

Recommendation

Consider accounting for every provisional PXT take until it is either burned or returned. Claim-based settlement may require persisting both the refund amount and an authenticated beneficiary, or using a design that does not provisionally take refundable PXT when the seller cannot be identified.

Team Update

The team has acknowledged that this is not a security issue and states:

Residual of RIFM. Exact-in burn leftover is refunded only in `attributeSell` *(ERC-20). ERC-6909 partial fills never hit that; orphan finalize books USDC only. Unrefunded PXT can sit on the hook (1.85% of unfilled). No authentic 6909 beneficiary.*

STCBT - Seed Transfers Can Be Taxed

Criticality	Minor / Informative
Location	PhoenixBuyback.sol#L191 Pxt.sol#L274
Status	Acknowledged

Description

`addLiquidity()` assumes that transferring `amount0Desired` and `amount1Desired` delivers those exact amounts to the collector. However, registering an address as `feeCollector` only exempts transfers where it is the sender. An inbound owner-to-collector PXT transfer remains subject to the 2.7% transfer fee unless the collector is separately assigned `FeeExempt` or `NoPenalty` status.

The function does not measure balance changes and proceeds with a caller-specified liquidity amount. An underfunded collector may therefore revert during PoolManager settlement, consume pre-existing PXT balances, or seed a different token composition than intended. Although the current bootstrap separately marks the collector as `FeeExempt`, this dependency is not enforced by the seeding path and may be missed during collector rotation or alternative deployments.

```
if (amount0Desired > 0) {
    IERC20(token0).safeTransferFrom(msg.sender, address(this),
amount0Desired);
}
if (amount1Desired > 0) {
    IERC20(token1).safeTransferFrom(msg.sender, address(this),
amount1Desired);
}

if (feeCollector != address(0) && from == feeCollector) {
    return PxtFeeModel.noFee(amount);
}
```

Recommendation

Consider deriving or validating the liquidity operation from the amounts actually received. The collector's required inbound-transfer exemption could also be established and verified as part of collector configuration rather than relying on a separate deployment step.

Team Update

The team has acknowledged that this is not a security issue and states:

Owner → collector PXT pays 2.7% unless the collector is `FeeExempt` *. Bootstrap always sets that before* `addLiquidity` *. Collector is one-shot; there is no rotation.*

VPBB - Valid Price Breaks Buybacks

Criticality	Minor / Informative
Location	PhoenixBuybackMath.sol#L17 PhoenixBuyback.sol#L193,209,309
Status	Acknowledged

Description

`pxtForQuote()` directly squares the `uint160` Uniswap square-root price using checked `uint256` arithmetic. Valid Uniswap prices can exceed `2^128`, meaning their square requires more than 256 bits. At or above this boundary, the multiplication reverts before `FullMath.mulDiv()` is reached.

If such a price is recorded and promoted as the frozen buyback reference, every buyback using it reverts during expected-output validation. For example, `sqrtPriceX96 = 2^128` is representable by `uint160`, but its square equals `2^256` and cannot fit in `uint256`.

```
function pxtForQuote(
    uint256 quoteAmount,
    uint160 sqrtPriceX96,
    bool pxtIsToken0
) internal pure returns (uint256 pxtAmount) {
    if (quoteAmount == 0 || sqrtPriceX96 == 0) return 0;

    uint256 priceX192 =
        uint256(sqrtPriceX96) * uint256(sqrtPriceX96);

    // Overflow occurs before FullMath is called.
    pxtAmount = FullMath.mulDiv(quoteAmount, priceX192, q192);
}
```

Recommendation

Consider using a full-precision price conversion that does not materialize the squared `uint160` value inside a single `uint256`. The implementation should support the complete valid Uniswap price domain and include boundary tests near the maximum supported square-root price.

Team Update

The team has acknowledged that this is not a security issue and states:

`pxtForQuote` squares `uint160` in `uint256`, so sqrt prices `≥ 2^128` revert and can freeze buybacks. That band is pathological for PXT/USDC (6/6). Does not mis-pay a fill.

Functions Analysis

Contract	Type	Bases		
	Function Name	**Visibility**	**Mutability**	**Modifiers**
V4Addresses	Library			
HookMiner	Library			
	find	Internal		
	computeAddress	Internal		
PhoenixV4ReturnDeltaHook	Implementation	BaseHook, Ownable, PxtFeeEvents, ISellAttributor		
		Public	✓	BaseHook Ownable
	setFeeCollector	External	✓	onlyOwner
	setOfficialPool	External	✓	onlyOwner
	sellProtectionCleared	External		-
	antiBotSeller	External		-
	getHookPermissions	Public		-
	flags	Public		-
	_beforeAddLiquidity	Internal		
	_afterAddLiquidity	Internal	✓	
	_beforeSwap	Internal	✓	

	_afterSwap	Internal	✓	
	_noteOfficialSpot	Internal	✓	
	creditFromPoolManager	External	✓	-
	pendingDexSellAmount	External		-
	consumeLpInbound	External	✓	-
	attributeSell	External	✓	-
	finalizeOrphanedSell	Public	✓	-
	_beforeSell	Internal	✓	
	_beforeBuy	Internal	✓	
	_afterSell	Internal	✓	
	_afterBuy	Internal	✓	
	_accrueUsdc	Internal	✓	
	_fairSellFeeBps	Internal		
	_applySellWindow	Internal	✓	
	_effectiveSellBalance	Internal		
	_pmInSlot	Internal		
	_enforceOfficialPool	Internal		
	_poolContainsPxt	Internal		
	_pxtIsCurrency0	Internal		
	_quoteCurrency	Internal		
	_classifySwap	Internal		
	_pxtIsSpecified	Internal		
	_recordPendingSkim	Internal	✓	

	_clearPendingSkim	Internal	✓	
	_sellUsdcSplit	Internal		
	_absDelta	Internal		
	_tstore	Internal	✓	
	_tload	Internal		
ISellAttributor	Interface			
	attributeSell	External	✓	-
	creditFromPoolManager	External	✓	-
	pendingDexSellAmount	External		-
	consumeLpInbound	External	✓	-
PhoenixFeeCollector	Implementation	PhoenixBuyback		
		Public	✓	PhoenixBuyback
	setHook	External	✓	onlyOwner
	receiveAccruedFees	External	✓	onlyHook
	collect	External	✓	nonReentrant
	_payoutToken	Internal	✓	
PhoenixBuybackMath	Library			
	pxtForQuote	Internal		
	enforceMinOut	Internal		
	sqrtPriceLimitAfterSlippage	Internal		

	floorToSpacing	Internal		
	recycleTicks	Internal		
	isPxtOnlyBand	Internal		
PhoenixBuyback	Implementation	IUnlockCallback, Ownable, AccessControl, ReentrancyGuard		
	_onlyHook	Internal		
		Public	✓	Ownable
	setAuthorizedBuybackCaller	External	✓	onlyRole
	_setAuthorizedBuybackCaller	Internal	✓	
	noteOfficialSpot	External	✓	onlyHook
	_promoteBuybackSpot	Internal	✓	
	_buybackRefSqrtPriceX96	Internal	✓	
	configurePool	External	✓	onlyOwner
	setBuybackParams	External	✓	onlyOwner
	addLiquidity	External	✓	onlyOwner nonReentrant
	_requireTradingOpen	Internal		
	executeBuyback	External	✓	nonReentrant
	_walletBurnDue	Internal		
	_executeBuybackCash	Internal	✓	
	pending	External		-
	quoteBuyback	External		-

	_reconcilePending	Internal	✓	
	unlockCallback	External	✓	-
	_burnToken	Internal	✓	
	_addRecycleLiquidity	Internal	✓	
IPxtSellControls	Interface			
	sellUnlockTimestamp	External		-
	antiBotSeller	External		-
	sellProtectionCleared	External		-
PxtSellAccess	Library			
	enforceSellUnlocked	Internal		
	enforceSellAccess	Internal		
	enforceTradingOpen	Internal		
PxtFeeEvents	Implementation			
PxtFeeModel	Library			
	noFee	Internal		
	grossUp	Internal		
	sellUsdcBps	Internal		
	splitBuy	Internal		
	splitSellBurn	Internal		

	splitSellUsdc	Internal		
Pxt	Implementation	ERC20, ERC20Permit, Ownable, AccessControl, PxtFeeEvents		
		Public	✓	ERC20 ERC20Permit Ownable
	decimals	Public		-
	sellUnlockTimestamp	Public		-
	setPoolManager	External	✓	onlyOwner
	setAntiBotSeller	External	✓	onlyOwner
	clearSellProtection	External	✓	-
	setFeeCollector	External	✓	onlyOwner
	setSellAttributor	External	✓	onlyOwner
	setApprovedContractRecipient	External	✓	onlyRole
	setWalletStatus	External	✓	onlyOwner
	quoteTransfer	External		-
	burnBalance	External	✓	-
	_setApprovedContractRecipient	Internal	✓	
	_isProtectedRecipient	Internal		
	_isWallet	Internal		
	_enforceContractRecipient	Internal		
	_enforceSellAccess	Internal		
	_update	Internal	✓	

	_onDexSell	Internal	✓
	_quoteTransfer	Internal	

Summary

Phoenix Token implements a fixed-supply ERC-20 ecosystem with Uniswap v4 trading fees, sell protections, dump penalties, USDC fee collection, and automated PXT buybacks. This audit investigates security issues, business logic concerns and potential improvements.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.

A **TAC Security** Company

The Cyberscope team

cyberscope.io